

▼ Mostrar más

En el ejemplo de arriba, `style={{}}` no es una sintaxis especial, sino un objeto regular `{}` dentro de las llaves de JSX de `style={ }`. Puedes utilizar el atributo `style` cuando tus estilos dependen de variables de JavaScript.

Renderizado condicional

En React, no hay una sintaxis especial para escribir condicionales. En cambio, usarás las mismas técnicas que usas al escribir código regular de JavaScript. Por ejemplo, puedes usar una sentencia `if` para incluir JSX condicionalmente:

```
let content;
if (isLoggedIn) {
  content = <AdminPanel />;
} else {
  content = <LoginForm />;
}
return (
  <div>
```

```
    {content}  
  </div>  
);
```

Si prefieres un código más compacto, puedes utilizar el [operador ? condicional](#). A diferencia de `if`, funciona dentro de JSX:

```
<div>  
  {isLoggedIn ? (  
    <AdminPanel />  
  ) : (  
    <LoginForm />  
  )}  
</div>
```

Cuando no necesites la rama `else`, puedes también usar la [sintaxis lógica `&&`](#), más breve:

```
<div>  
  {isLoggedIn && <AdminPanel />}  
</div>
```

Todos estos enfoques también funcionan para especificar atributos condicionalmente. Si no estás familiarizado con toda esta sintaxis de JavaScript, puedes comenzar por usar siempre `if...else`.

Renderizado de listas

Dependerás de funcionalidades de JavaScript como los [bucles `for`](#) y la [función `map\(\)` de los arreglos](#) para renderizar listas de componentes.

Por ejemplo, digamos que tienes un arreglo de productos:

```
const products = [
```

```
{ title: 'Col', id: 1 },  
{ title: 'Ajo', id: 2 },  
{ title: 'Manzana', id: 3 },  
];
```

Dentro de tu componente, utiliza la función `map()` para transformar el arreglo de productos en un arreglo de elementos `<li>`:

```
const listItems = products.map(product =>  
  <li key={product.id}>  
    {product.title}  
  </li>  
);  
  
return (  
  <ul>{listItems}</ul>  
);
```

Nota que `<li>` tiene un atributo `key` (llave). Para cada elemento en una lista, debes pasar una cadena o un número que identifique ese elemento de forma única entre sus hermanos. Usualmente, una llave debe provenir de tus datos, como un ID de una base de datos. React dependerá de tus llaves para entender qué ha ocurrido si luego insertas, eliminas o reordenas los elementos.

App.js

⬇ Descargar ↺ Reiniciar ↗

```
const products = [  
  { title: 'Col', isFruit: false, id: 1 },  
  { title: 'Ajo', isFruit: false, id: 2 },  
  { title: 'Manzana', isFruit: true, id: 3 },  
];  
  
export default function ShoppingList() {  
  const listItems = products.map(product =>  
    <li  
      key={product.id}  
      style={{
```

```
color: product.isFruit ? 'magenta' : 'darkgreen'
```

▼ Mostrar más

Responder a eventos

Puedes responder a eventos declarando funciones *controladoras de eventos* dentro de tus componentes:

```
function MyButton() {  
  function handleClick() {  
    alert('¡Me hiciste clic!');  
  }  
  
  return (  
    <button onClick={handleClick}>  
      Hazme clic  
    </button>  
  );  
}
```

¡Nota que `onClick={handleClick}` no tiene paréntesis al final! No *llames* a la función controladora de evento: solamente necesitas *pasarla hacia abajo*. React llamará a tu controlador de evento cuando el usuario haga clic en el botón.

Actualizar la pantalla

A menudo, querrás que tu componente “recuerde” alguna información y la muestre. Por ejemplo, quizá quieras contar el número de veces que hiciste clic en un botón. Para lograrlo, añade *estado* a tu componente.

Primero, importa `useState` de React:

```
import { useState } from 'react';
```

Ahora puedes declarar una *variable de estado* dentro de tu componente:

```
function MyButton() {  
  const [count, setCount] = useState(0);  
  // ...  
}
```

Obtendrás dos cosas de `useState`: el estado actual (`count`), y la función que te permite actualizarlo (`setCount`). Puedes nombrarlos de cualquier forma, pero la convención es llamarlos algo como `[something, setSomething]`.

La primera vez que se muestra el botón, `count` será `0` porque pasaste `0` a `useState()`. Cuando quieras cambiar el estado llama a `setCount()` y pásale el nuevo valor. Al hacer clic en este botón se incrementará el contador:

```
function MyButton() {  
  const [count, setCount] = useState(0);  
  
  function handleClick() {  
    setCount(count + 1);  
  }  
}
```